

FPGA-Based Implementation and Verification of the PRESENT-80 Lightweight Block Cipher on a Gowin GW5A Device

Tran Tuan Minh

Undergraduate Student of Integrated Circuit Engineering

FPT University HCM

Student ID: SE203134

Email: tuanminhbl12@gmail.com

Abstract—This paper presents the design, implementation, and verification of a PRESENT-80 lightweight block cipher on a Gowin GW5A-LV25UG324C2/I1 field-programmable gate array (FPGA). The encryption core was described in Verilog HDL using an iterative round-based architecture with a 64-bit state register, an 80-bit on-the-fly key schedule, sixteen parallel 4-bit S-boxes, and a combinational permutation layer. A UART interface was added so that a personal computer can send test commands, plaintext, and key values to the FPGA and receive the resulting ciphertext. The design was functionally checked in ModelSim and compared with an independent Python golden model. Physical verification was performed on the ACG525 development board through a CH9102 USB-to-UART interface. For the known all-zero plaintext and all-zero key vector, both the FPGA and the golden model produced the ciphertext 5579C1387B228445. Post-place-and-route results show that the complete design uses 777 lookup tables and 446 registers. The implementation meets the 50 MHz timing requirement, with an estimated maximum frequency of 160.038 MHz and no reported setup or hold violations.

Index Terms—FPGA, lightweight cryptography, PRESENT-80, Verilog HDL, UART, golden model, Gowin GW5A, ModelSim

I. INTRODUCTION

The number of small connected devices has increased in embedded, sensor, identification, and Internet of Things applications. These devices often have limited logic area, memory, energy, and processing capability. NIST has noted that algorithms designed mainly for general-purpose computers may not always provide acceptable performance in constrained environments, which motivates the study of lightweight cryptography [1]. In this context, “lightweight” refers to implementation cost and efficiency; it does not mean that security can be ignored.

PRESENT was introduced as a compact block cipher for strongly resource-constrained hardware [2]. It operates on a 64-bit block and supports 80-bit or 128-bit keys. PRESENT is also included in ISO/IEC 29192-2 as a block cipher for lightweight cryptographic implementations [3]. Its regular substitution-permutation structure is suitable for learning the relationship between a cryptographic algorithm and an RTL architecture.

Modern lightweight cryptography also includes authenticated encryption schemes. For example, NIST standardized the Ascon family for authenticated encryption, hashing, and extendable-output functions in 2025 [4]. Therefore, PRESENT-80 should not be presented as a replacement for every modern security protocol. In this work it is used as an educational and hardware-design case study: the main goal is to construct a correct FPGA implementation, connect it to a host computer, and verify it using independent reference results.

The main contributions of this project are:

- an iterative Verilog implementation of PRESENT-80 encryption;
- an on-the-fly round-key generator that avoids storing all round keys;
- a UART command interface for self-test and user-selected plaintext/key values;
- an independent Python golden model for reference ciphertext generation;
- ModelSim verification and physical testing on a Gowin ACG525 board; and
- a post-place-and-route evaluation using resource and timing reports from Gowin EDA Education.

The remainder of this paper is organized as follows. Section II gives the technical background and related work. Section III describes the PRESENT-80 algorithm. Section IV presents the proposed hardware architecture. Section V explains UART integration and board constraints. Section VI describes the verification method. Section VII presents the experimental setup and results. Section VIII discusses limitations and future improvements, and Section IX concludes the paper.

II. BACKGROUND AND RELATED WORK

A. Lightweight Cryptography

A cryptographic design for a constrained device must balance security, logic area, memory, latency, throughput, and energy. The best design is not determined by only one number. A fully parallel architecture may provide high throughput but use more logic, while a serialized architecture may save area

but require more cycles. Hardware comparisons must also consider device family, synthesis tool, process technology, clock constraint, and whether communication logic is included [5].

NISTIR 8114 describes the motivation for studying algorithms intended for constrained devices [1]. More recently, NIST standardized Ascon-based primitives for modern lightweight authenticated encryption and hashing [4]. These developments show that lightweight cryptography is an active engineering topic rather than only an academic idea.

B. PRESENT in Hardware

The original PRESENT paper reports a compact hardware objective and describes a 31-round substitution-permutation network [2]. Poschmann later discussed multiple hardware implementation strategies, including FPGA implementations and the trade-off between serialized and round-based structures [6]. Prior work also shows that an architecture that is compact in an ASIC is not automatically optimal on an FPGA, because FPGA lookup tables, routing, registers, and embedded memories have different cost structures [7].

The design in this paper is not intended to claim a new cryptographic algorithm. Its contribution is a complete, reproducible student-level implementation flow on a Gowin FPGA platform, including the cipher core, UART interface, simulation, Python reference model, timing constraints, and physical demonstration.

III. PRESENT-80 ALGORITHM

A. Algorithm Parameters

PRESENT-80 encrypts a 64-bit plaintext block using an 80-bit key. The cipher contains 31 rounds followed by a final round-key addition. Let S_i denote the 64-bit state before round i , and let K_i denote the 64-bit round key obtained from the most significant bits of the 80-bit key register. A round can be written as

$$S_{i+1} = P(S(S_i \oplus K_i)), \quad 1 \leq i \leq 31, \quad (1)$$

where $S(\cdot)$ is the S-box layer and $P(\cdot)$ is the bit-permutation layer. After round 31, the final ciphertext is

$$C = S_{32} \oplus K_{32}. \quad (2)$$

B. S-box Layer

The state is divided into sixteen 4-bit nibbles. The same 4-bit S-box is applied to every nibble. The PRESENT S-box is shown in Table I. In the proposed RTL design, all sixteen S-boxes operate in parallel as combinational logic.

TABLE I
PRESENT 4-BIT S-BOX

x	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
$S(x)$	C	5	6	B	9	0	A	D	3	E	F	8	4	7	1	2

C. Permutation Layer

The permutation layer moves individual state bits to fixed destinations. For bit positions $0 \leq i < 63$, the mapping is

$$P(i) = 16i \bmod 63, \quad (3)$$

while bit 63 remains at position 63. Since this operation contains no arithmetic data processing, it is implemented mainly by fixed wiring and routing in RTL.

D. Key Schedule

The 80-bit key register is updated after each round by three steps:

- 1) rotate the key register left by 61 bit positions;
- 2) apply the PRESENT S-box to bits [79 : 76]; and
- 3) XOR the 5-bit round counter into bits [19 : 15].

The round key is selected as bits [79 : 16] of the key register. The proposed architecture generates this key on the fly, so a memory for all 32 round keys is not required.

IV. PROPOSED FPGA ARCHITECTURE

A. System Overview

Fig. 1 shows the complete prototype. The personal computer communicates with the FPGA through UART. The command parser loads the plaintext and key, starts the cipher core, and formats the 64-bit ciphertext as readable hexadecimal characters.

B. Iterative Encryption Core

The encryption core uses a round-based iterative datapath. Its main registers are:

- a 64-bit state register;
- an 80-bit key register;
- a 5-bit round counter; and
- a 64-bit ciphertext output register.

One PRESENT round is evaluated per active clock cycle. The AddRoundKey, S-box layer, permutation layer, and key update are combinational operations between registers. The state and key are updated at the next rising edge. This structure is easier to understand and uses fewer replicated round blocks than a fully unrolled 31-round implementation.

The core provides a simple handshake:

- **start**: accepts a new plaintext and key when idle;
- **busy**: indicates that round processing is active;
- **done**: produces a one-cycle completion pulse; and
- **ready**: indicates that the core can accept a request.

In the current RTL, the round logic operates for 31 clock cycles after a request is accepted. At 50 MHz, one clock period is 20 ns, so the round-processing time is nominally

$$t_{\text{rounds}} = 31 \times 20 \text{ ns} = 620 \text{ ns}. \quad (4)$$

The full start-to-start interval and system latency must be measured from the implemented waveform because the wrapper FSM adds control and UART time.

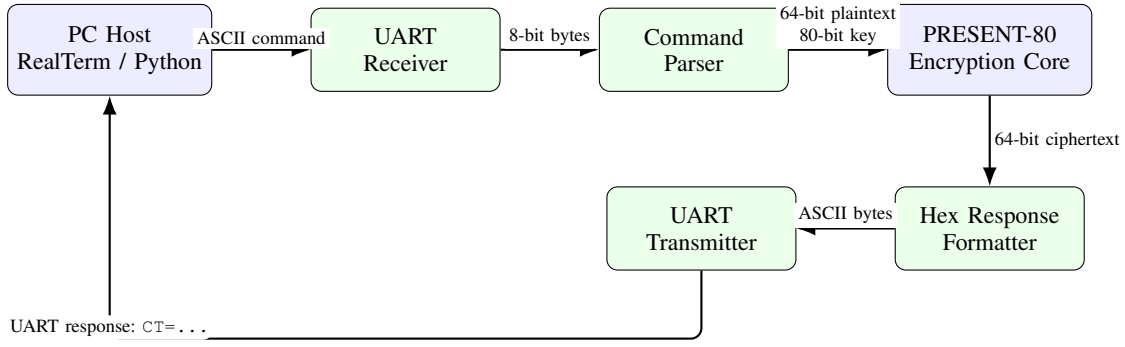


Fig. 1. System-level architecture of the FPGA-based PRESENT-80 prototype.

TABLE II
ASCII UART COMMANDS

Command	Function
T	Runs a built-in self-test with zero plaintext and zero key.
E	Receives 16 hexadecimal plaintext characters followed by 20 hexadecimal key characters. Spaces, tab, carriage return, and line feed may be ignored by the parser.

TABLE III
BOARD-LEVEL PIN ASSIGNMENTS

Signal	Function	FPGA Pin
clk	50 MHz clock	T9
rst_n	Active-low reset / KEY0	B16
uart_tx	FPGA-to-PC UART	V8
uart_rx	PC-to-FPGA UART	U8
led[0]	Status LED	D14
led[1]	Status LED	C14
led[2]	Status LED	B9
led[3]	Status LED	A9

C. Top-Level Control FSM

The final board wrapper contains states for:

- 1) transmitting a READY banner after reset;
- 2) waiting for a command;
- 3) receiving hexadecimal input;
- 4) starting the PRESENT core;
- 5) waiting for encryption completion;
- 6) transmitting the ciphertext response; and
- 7) reporting an input-format error.

LED outputs are used as simple status indicators for core activity, completion, received-byte activity, and command errors.

V. UART INTERFACE AND BOARD INTEGRATION

A. Command Protocol

UART only transfers bytes; it does not define the meaning of the received data. Therefore, a small application-level command protocol was designed. The final hardware demonstration uses 115200 baud, 8 data bits, no parity, and 1 stop bit.

The custom encryption format is

```
E <16 hexadecimal plaintext characters>
  <20 hexadecimal key characters>
```

For example,

```
E 0000000000000000 00000000000000000000
```

requests encryption of a 64-bit all-zero block with an 80-bit all-zero key. The FPGA response format is

```
CT=5579C1387B228445
```

ASCII hexadecimal was selected instead of raw binary because it is easy to type, display, capture in screenshots, and compare with the Python golden model. The disadvantage is communication overhead: 36 hexadecimal data characters are transmitted instead of 18 raw data bytes. This overhead is acceptable for a teaching and verification prototype, where clarity is more important than communication throughput.

B. Board and Pin Constraints

The target platform is the ACG525 development board using a Gowin GW5A-LV25UG324C2/I1 FPGA. The board provides a 50 MHz clock and a USB-to-UART connection. The important physical pin assignments are listed in Table III [8], [9].

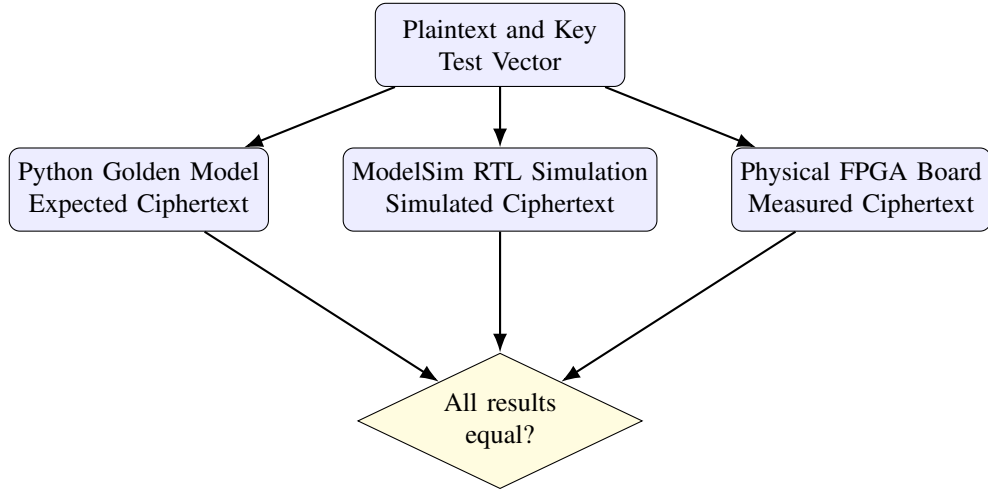
The physical constraint file assigns these ports using the LVCMOS33 I/O standard. Pull-up settings are applied to the active-low reset and UART receive input. The timing constraint declares a 20 ns period for the 50 MHz system clock:

```
create_clock -name clk_50m -period 20.000 \
[get_ports {clk}]
```

This constraint allows the implementation tool to evaluate whether the state, key-schedule, UART, and control paths meet the required clock period.

VI. VERIFICATION METHODOLOGY

The project was checked at three levels as shown in Fig. 2. Using multiple verification levels reduces the chance that a single shared mistake will remain unnoticed.



PASS if equal; otherwise debug the mismatch

Fig. 2. Three-level verification flow used in the project.

A. Python Golden Model

The golden model implements PRESENT-80 independently in Python. It contains separate functions for the S-box layer, permutation layer, key update, and complete encryption. The command

```
py present80_golden.py \
  000000000000000000 \
  00000000000000000000
```

produces

```
PT=000000000000000000
K =00000000000000000000
CT=5579C1387B228445
```

The golden model is intentionally not generated from the Verilog code. This independence is important: if the RTL and reference model copied the same error, matching outputs would not provide strong evidence of correctness.

B. ModelSim Verification

The cipher-core testbench applies reset, presents the all-zero plaintext/key pair, pulses `start`, waits for `done`, and checks the ciphertext automatically. The test reports a pass only if the output equals 5579C1387B228445.

A system-level testbench also exercises the UART receiver, wrapper control, PRESENT core, and UART transmitter. For simulation speed, the UART baud parameter may be increased while preserving the same frame format. The paper must clearly distinguish this simulation-only baud value from the 115200-baud physical-board configuration.

C. Physical Board Verification

After synthesis, place and route, and bitstream generation, the configuration file is programmed onto the ACG525 board. RealTerm or a Python serial program opens the CH9102 COM port at 115200 8-N-1. The test procedure is:

TABLE IV
EXPERIMENTAL SETUP

Item	Configuration
FPGA board	ACG525 development board
FPGA device	GW5A-LV25UG324C2/I1
System clock	50 MHz
HDL	Verilog HDL
Synthesis / P&R	Gowin EDA Education 1.9.11.01 Education
Simulation	ModelSim INTEL FPGA STARTER EDITION 2020.1
Golden model	Python Antigravity IDE Version: 2.1.1 — Python 3.13.14
Board terminal	RealTerm
UART	115200 baud, 8-N-1, no flow control

- 1) reset the board and confirm the `READY` response;
- 2) send `T` for the built-in known-vector test;
- 3) send one or more `E` commands;
- 4) record the returned `CT=` string; and
- 5) compare the result with the Python golden model.

VII. EXPERIMENTAL SETUP AND RESULTS

A. Development Environment

Table IV summarizes the implementation environment. Add the exact software versions before submission.

B. Functional Results

Table V lists reference ciphertexts generated by the Python model. The first vector was confirmed on the physical board. The remaining FPGA columns must be completed using measured board outputs rather than copied expected values.

C. Resource Utilization

Resource data must be copied from the final post-place-and-route report, not from an earlier synthesis run. The utilization

TABLE V
GOLDEN-MODEL AND FPGA VERIFICATION RESULTS

Test	Plaintext (64 bit)	Key (80 bit)	Golden-Model Ciphertext	FPGA Ciphertext	Result
1	0000000000000000	00000000000000000000	5579C1387B228445	5579C1387B228445	PASS
2	FFFFFFFFFFFFFFF	00000000000000000000	A112FFC72F68417B	A112FFC72F68417B	PASS
3	0000000000000000	FFFFFFFFFFFFFFF	E72C46C0F5945049	E72C46C0F5945049	PASS
4	FFFFFFFFFFFFFFF	FFFFFFFFFFFFFFF	3333DCD3213210D2	3333DCD3213210D2	PASS

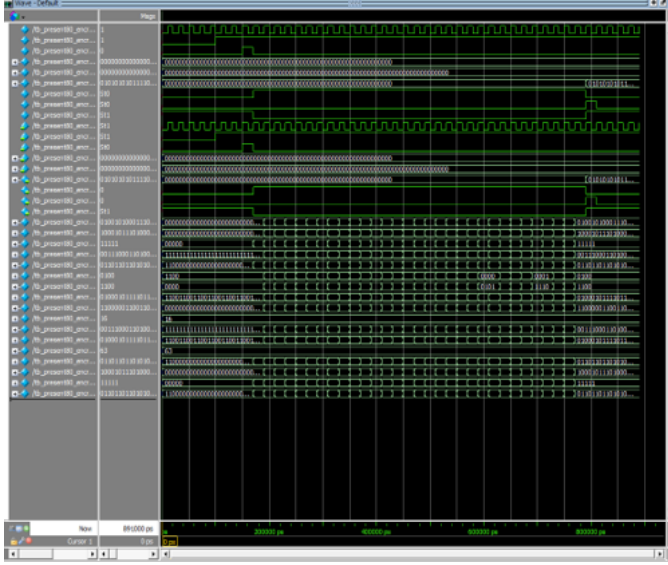


Fig. 3. ModelSim waveform of the PRESENT-80 encryption core. The waveform shows the system clock, reset, start request, round processing, ciphertext output, and completion signal.

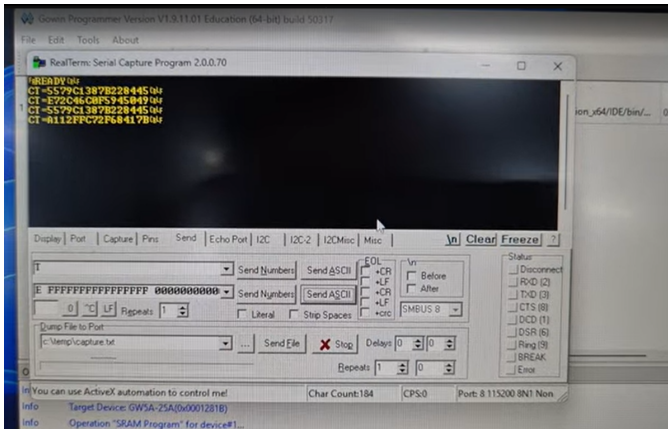


Fig. 4. UART response obtained from the physical ACG525 FPGA board through RealTerm. The returned ciphertext matches the Python golden-model result.

TABLE VI
POST-PLACE-AND-ROUTE RESOURCE UTILIZATION

Resource	Used	Available	Utilization
LUTs	777	23040	3.37%
Total logic	781	23040	3.39%
Registers / FFs	446	23685	1.88%
I/O ports	8	222	3.60%
RAM16 / SSRAM	0	—	0%
DSP blocks	0	—	0%
PLL	0	—	0%

TABLE VII
POST-ROUTE TIMING SUMMARY

Metric	Measured Value
Target frequency	50 MHz
Target clock period	20.000 ns
Estimated maximum frequency	160.038 MHz
Approx. critical-path delay	6.249 ns
Setup violations	0
Hold violations	0
Total negative slack	0.000 ns
Timing result	PASS

of the complete top-level design should be reported because the UART and command parser are part of the physical prototype.

Because PRESENT mainly uses XOR gates, small lookup tables, registers, and fixed permutations, BSRAM and DSP use are expected to be low or zero. This expectation must still be confirmed using the actual report.

D. Timing Results

The timing constraint is 20 ns. The most important timing values are the worst setup slack, the critical-path delay, and the final pass/fail status.

If WNS is positive, the design meets the 50 MHz setup requirement. If it is negative, the report should not claim successful timing closure. Instead, the critical path should be identified and optimized.

E. Latency and Throughput

Let N_{core} be the measured number of cycles between the accepted start request and the completion pulse. Core latency is

$$L_{\text{core}} = \frac{N_{\text{core}}}{f_{\text{clk}}}. \quad (5)$$

If the core accepts one block every N_{interval} cycles, the block-processing throughput is

$$T_{\text{core}} = \frac{64f_{\text{clk}}}{N_{\text{interval}}} \text{ bit/s.} \quad (6)$$

The ModelSim waveform shows that the PRESENT-80 core completes one encryption in 31 clock cycles. At the 50 MHz board clock, this corresponds to a latency of 620 ns. With a 32-cycle initiation interval, the theoretical core throughput is 100 Mbit/s. UART transmission time is not included in this value.

VIII. DISCUSSION

The confirmed all-zero test demonstrates agreement among the known PRESENT result, the Python model, the RTL simulation, and the physical FPGA output. Additional nonzero tests are important because an all-zero-only test may fail to reveal some bit-order, S-box-index, or key-schedule errors. The final paper should therefore include several measured nonzero vectors from Table V.

The iterative architecture provides a clear area–latency trade-off. Only one round datapath is implemented and reused 31 times. A fully unrolled design could reduce the number of cycles per block but would replicate round logic and increase routing complexity. A more serialized design could reduce combinational logic but would require more cycles. This project selects the round-based structure because it is suitable for a second-year FPGA course: the datapath is visible, the FSM is manageable, and the performance is sufficient for UART demonstration.

The ASCII UART protocol improves usability but should not be confused with an optimized cryptographic data interface. For high-volume encryption, a binary packet format, FIFO buffering, SPI, USB, or Ethernet would be more efficient. The protocol used here is intended for observation and functional verification.

Finally, successful ciphertext matching does not prove resistance against physical attacks. The current implementation has no masking, hiding, fault detection, secure key storage, or authenticated encryption. These are important considerations for a real security product.

IX. LIMITATIONS AND FUTURE WORK

The current prototype has the following limitations:

- encryption only; decryption is not implemented;
- PRESENT-80 only; PRESENT-128 is not included;
- one 64-bit block per request;
- no block mode such as CBC or CTR;
- no message authentication or AEAD;
- no power measurement or side-channel evaluation;
- no protected key storage; and
- an ASCII UART interface intended for testing rather than high throughput.

Future work may add PRESENT-128, decryption, CTR mode, FIFO-based binary communication, and an automated Python application that sends many random test vectors and

compares every FPGA result with the golden model. Another useful extension is to compare the PRESENT core with a small AES or Ascon implementation under the same FPGA, tool version, clock constraint, and interface conditions. Such a controlled comparison would be more meaningful than copying results from different devices and synthesis tools.

X. CONCLUSION

This work developed a complete FPGA-based PRESENT-80 encryption prototype on a Gowin GW5A device. The project connected algorithm-level operations to a practical RTL architecture consisting of an iterative cipher core, key schedule, command parser, UART receiver, and UART transmitter. Correctness was checked using an independent Python golden model, ModelSim, and a physical ACG525 board.

For the all-zero plaintext and all-zero key test, the implementation returned 5579C1387B228445, matching the reference model. The remaining experimental contribution is completed by inserting the measured resource utilization, timing summary, and additional hardware test vectors into the provided tables. With those measurements, the paper gives a reproducible example of lightweight-cryptography design, verification, and FPGA implementation suitable for an introductory semiconductor or integrated-circuit engineering project.

REFERENCES

- [1] K. McKay, L. Bassham, M. S. Turan, and N. Mouha, “Report on lightweight cryptography,” National Institute of Standards and Technology, Tech. Rep. NISTIR 8114, 2017.
- [2] A. Bogdanov, L. R. Knudsen, G. Leander, C. Paar, A. Poschmann, M. J. B. Robshaw, Y. Seurin, and C. Vikkelsøe, “PRESENT: An ultra-lightweight block cipher,” in *Cryptographic Hardware and Embedded Systems – CHES 2007*, ser. Lecture Notes in Computer Science, vol. 4727. Springer, 2007, pp. 450–466.
- [3] International Organization for Standardization, “Information security—lightweight cryptography—part 2: Block ciphers,” ISO/IEC, Tech. Rep. ISO/IEC 29192-2:2019, 2019.
- [4] M. S. Turan, K. McKay, J. Kang, J. Kelsey, and D. Chang, “Ascon-based lightweight cryptography standards for constrained devices: Authenticated encryption, hash, and extendable output functions,” National Institute of Standards and Technology, Tech. Rep. NIST Special Publication 800-232, 2025.
- [5] S. Kerckhof, F. Durvaux, C. Hocquet, D. Bol, and F.-X. Standaert, “Towards green cryptography: A comparison of lightweight ciphers from the energy viewpoint,” in *Cryptographic Hardware and Embedded Systems – CHES 2012*, ser. Lecture Notes in Computer Science, vol. 7428. Springer, 2012, pp. 390–407.
- [6] A. Y. Poschmann, “Lightweight cryptography: Cryptographic engineering for a pervasive world,” Ph.D. dissertation, Ruhr-University Bochum, 2009, iACR Cryptology ePrint Archive, Report 2009/516.
- [7] S. Kolay and D. Mukhopadhyay, “Khudra: A new lightweight block cipher for FPGAs,” in *Security, Privacy, and Applied Cryptography Engineering*, ser. Lecture Notes in Computer Science, vol. 8804. Springer, 2014, pp. 126–145.
- [8] Gowin Semiconductor Corporation, *GW5A Series of FPGA Products Data Sheet*, Gowin Semiconductor Corporation, 2025.
- [9] Xiaomeige FPGA Team, *Pin Information Table of the ACG525 Gowin FPGA Development Board*, Wuhan Xinluheng Technology, 2025, board pin-assignment document supplied with the development kit.